

ICT ACADEMY OF KERALA

Full Stack Application Development with ReactJS



Akash V A

Anvin M V

ICT Academy of Kerala

30.07.2024

EXECUTIVE SUMMARY

The **Food Ordering System** is a modern, full-stack web application designed to streamline online food ordering, menu exploration, shopping cart management, and administrative store operations. Developed as part of the **Full Stack Application Development with ReactJS** program at the **ICT Academy of Kerala**, this project addresses the growing demand for fast, intuitive, and responsive digital food ordering solutions.

The application leverages a cutting-edge MERN stack architecture comprising **React 19**, **Vite 8**, **Tailwind CSS v4**, **Node.js**, **Express.js**, and **MongoDB with Mongoose ODM**. The client interface features single-page navigation via **React Router v7**, global state management with React Context API (Cart, Auth, Theme, Toast), and a responsive component design system. On the backend, a modular RESTful API provides secure authentication using **JSON Web Tokens (JWT)** and **Bcrypt.js**, role-based authorization for customers and administrators, and persistent CRUD management for foods, categories, user profiles, and order status workflows.

INTRODUCTION

In the contemporary digital service economy, online food delivery platforms have shifted from a consumer luxury to an essential everyday utility. Traditional paper menus and phone-in orders are increasingly replaced by real-time digital interfaces that offer dynamic visual menus, instant customized carts, transparent pricing, and live status updates.

The objective of this project is to construct a production-grade, full-stack food ordering platform that bridges customer convenience with store administration efficiency. Built within the context of the ICT Academy of Kerala's internship program, the project provides hands-on implementation of full-stack engineering principles, modern React 19 component patterns, asynchronous state handling, REST API design, and MongoDB database modeling.

OBJECTIVES

Clearly state the objectives of your internship project, outlining what you aimed to achieve and the skills you intended to develop or enhance during the internship:

1. **Build a Responsive Frontend UI:** Develop an intuitive, mobile-friendly interface using React 19, Vite, and Tailwind CSS v4 with support for dynamic dark/light themes.
2. **Implement RESTful Backend Services:** Design modular Express.js routes and controllers for managing foods, categories, user accounts, orders, and system settings.
3. **Establish Secure Authentication & Role-Based Access (RBAC):** Implement JWT-based authentication with password encryption (Bcrypt.js) for separate Customer and Admin access levels.

4. **Architect Global State Management:** Utilize React Context API (AuthContext, CartContext, ThemeContext, ToastContext) to handle user sessions, shopping cart updates, notifications, and application themes cleanly.
5. **Develop Customer Ordering Workflow:** Enable multi-criteria menu filtering (category, search, price range, sorting), food item details view, cart manipulations, and structured checkout with order placement.
6. **Construct Admin Control Center:** Build a dedicated administrative portal featuring high-level store metrics, food item CRUD operations, customer order status updates, and customer list auditing.

SCOPE AND DELIVERABLES

Define the scope of your project, detailing the specific tasks, activities, and deliverables that you were responsible for during the internship:

- **Client Single Page Application (SPA):** Built with React 19, React Router v7, and Tailwind CSS v4 containing 14 page views and modular UI component libraries.
- **Server RESTful Web Service:** Node.js & Express server hosting API endpoints under `/api/auth`, `/api/foods`, `/api/categories`, `/api/orders`, and `/api/settings`.
- **Database Schema & ODM Layer:** Mongoose schemas for User, Food, Category, Order, and Settings models with strict validation rules.
- **Database Seeding Tool:** Automated script (`seed.js`) for populating initial food categories, catalog items, admin access credentials, and sample customer orders.
- **Security Middleware System:** JWT verification and role-guard middleware for protecting admin endpoints and user profile routes.

METHODOLOGY

Describe the approach or methodology you followed to execute your project, including any specific techniques, tools, or frameworks that you utilized:

The development was executed following an **Agile Iterative Software Engineering Methodology**, structured into modular sprints focusing on component design, database setup, API development, integration, and refinement.

Layer	Technology	Role / Purpose
Frontend Framework	React 19 + Vite 8	High-performance Single Page Application with fast HMR
Routing	React Router v7	Declarative browser routing, layouts, and route guards
Styling System	Tailwind CSS v4 + React Icons	Utility-first, responsive CSS layout and modern icon set
State Management	React Context API	Global providers for Auth, Cart, Theme, and Toasts
Backend Runtime	Node.js + Express.js	Asynchronous RESTful Web Service framework
Database & ODM	MongoDB + Mongoose 8	NoSQL document database with typed schema modeling
Security	JWT + Bcrypt.js	Stateless authentication tokens and password hashing

PROJECT ACTIVITIES

Provide a detailed breakdown of the activities you undertook during your internship, describing the tasks you performed, the tools or technologies you used, and any challenges or obstacles you encountered:

- 1. Database Schema Design:** Configured Mongoose schemas for users, categories, foods, orders, and store settings with relational references.
- 2. REST API Engineering:** Formatted route handlers for customer authentication, food filtering, order creation, order tracking, and admin dashboard operations.
- 3. Frontend Component & Page Architecture:** Built 14 dedicated user screens across customer storefront and admin management portal.

4. Global State Pipeline: Implemented Context API hooks to manage cart additions, token storage, light/dark themes, and toast notifications.

RESULTS & FINDINGS

Present the results and findings of your internship project, highlighting any key accomplishments, insights, or data analysis that you conducted:

Route Path	View / Component	Access Level	Description
/	HomePage	Public	Hero banner, categories, featured items, offers
/menu	MenuPage	Public	Full food catalog with search, filters, & sorting
/food/:id	FoodDetailsPage	Public	Food specifications, ratings, reviews, & cart add
/cart	CartPage	Customer	Cart item list, quantity controls, subtotal summary
/checkout	CheckoutPage	Customer (Auth)	Address input, payment selection, & order placement
/orders	OrdersPage	Customer (Auth)	User's order history with status timeline tags
/admin/dashboard	AdminDashboardPage	Admin Only	Store summary, total revenue, order count charts
/admin/foods	AdminFoodsPage	Admin Only	Food item CRUD modal, stock toggle, price edit
/admin/orders	AdminOrdersPage	Admin Only	Order fulfillment table & status transition selector

CONCLUSION

Summarize the main outcomes and learnings from your internship project, reflecting on how well you achieved your objectives.

The development of the **Food Ordering System** successfully satisfies all project requirements and program objectives set forth by the **ICT Academy of Kerala**. The project demonstrates the execution of modern web development using the MERN stack (MongoDB, Express, React, Node.js) paired with Vite 8, React 19, and Tailwind CSS v4.

APPENDIX

Include any additional supporting documents, samples of your work, project artifacts, or reference materials that are relevant to your internship project:

A. User & Order Mongoose Schemas

```
// User Schema
{
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['customer', 'admin'], default: 'customer' }
}

// Order Schema
{
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  orderItems: [{ food: mongoose.Schema.Types.ObjectId, quantity: Number, price: Number }],
  totalPrice: Number,
  status: { type: String, enum: ['Pending', 'Preparing', 'Out for Delivery', 'Delivered', 'Cancel']
}
```

B. Setup & Run Instructions

```
# Install dependencies
npm install && cd server && npm install

# Seed database & Start server
npm run seed
npm run server (Port 5000)
npm run dev    (Port 5173)
```